

synthesis

Relatório técnico do frontend

Explicação direta, sem espuma, para quem vem do backend e quer entender onde estão as peças, como os dados circulam e onde encaixar a futura API Django.

Gerado em 23/08/2026.

1. Resumo para cérebro cansado

Este projeto é um frontend Next.js para a aplicação interna **synthesis**. Ele não é um kanban, não mede produtividade e não tenta virar ferramenta de gestão de tarefas. A missão dele é simples:

- gestores registram atividades reais feitas na semana;
- a gerente escolhe quais relatos entram no relatório;
- o sistema monta um rascunho visual em mosaico;
- a gerente edita somente o card editorial;
- o relato original continua intacto;
- o sistema gera versões de PDF para histórico.

Pense assim: `ActivityReport` é o dado original, tipo uma tabela imutável de entrada.
`ReportCard` é a cópia editorial usada no relatório, tipo uma view materializada editável.

2. Stack usada

Framework

`Next.js` com App Router, React e TypeScript strict.

UI

`Tailwind CSS`, componentes locais no estilo `shadcn/ui` e ícones `Lucide`.

Formulário

`React Hook Form` com validação via `Zod`.

Dados assíncronos

`TanStack Query` para cache e estado de carregamento.

Editor

`dnd-kit` para reordenar cards do relatório.

Imagem

`react-easy-crop` para a tela de recorte.

3. Como rodar sem drama

```
npm install
npm run dev
```

Depois abra `http://localhost:3000`. Se quiser usar outra porta:

```
npm run dev -- -p 3001
```

Comandos de validação

```
npm run typecheck
npm run lint
npm test
npm run build
```

4. Estrutura mental do projeto

```
src/  
  app/           rotas do Next App Router  
  components/    peças reutilizáveis de tela  
  components/activity/ relatos e formulário de atividade  
  components/report/  seleção, editor, canvas A4 e PDF  
  components/layout/  menu lateral, cabeçalho e shell  
  services/        camada de API e mocks  
  types.ts        tipos centrais do domínio  
  lib/            utilitários pequenos  
  test/           testes
```

Se você veio do backend, leia nesta ordem:

- `src/types.ts` : entidades do domínio.
- `src/services/api.ts` : contrato da API que o Django deve implementar depois.
- `src/services/mock-data.ts` : dados fake usados agora.
- `src/app/...` : páginas e rotas.
- `src/components/...` : pedaços visuais reutilizáveis.

5. As entidades que importam

ActivityReport

É o relato original enviado pelo gestor. Contém título, data, local, descrição, resultado, público beneficiado, área, gestor e fotos.

Regra de ouro: edição editorial não mexe aqui.

ReportCard

É a versão editorial do relato dentro do relatório. Pode ter título resumido, texto editado, resultado adaptado, foto escolhida e ordem manual.

Regra de ouro: mexer no card não altera o relato original.

WeeklyReport

Representa o relatório semanal. Guarda atividades selecionadas, seções por área, cards e versões de PDF geradas.

ReportVersion

Cada geração de PDF cria uma nova versão. Versões antigas não são sobrescritas visualmente.

Se no backend você fizer um endpoint que atualiza `ActivityReport` quando a gerente edita `ReportCard`, o sistema fica conceitualmente errado. É tipo dar UPDATE na nota fiscal porque alguém corrigiu o texto do e-mail.

6. Fluxo principal

1. Gestor registra atividade

Rota: `/relatos/novo` . O formulário valida campos obrigatórios e exige foto principal. O gestor e a área vêm preenchidos a partir do usuário autenticado.

2. Gerente seleciona relatos

Rota: `/relatorio-semanal/selecao` . A gerente marca quais relatos entram no relatório. Nada selecionado significa botão de rascunho desabilitado.

3. Sistema gera rascunho

A função `generateDraft` cria um `WeeklyReport` com seções por área e cards baseados nos relatos selecionados.

4. Gerente edita mosaico

Rota: `/relatorio-semanal/[id]/editar` . A tela tem três colunas:

- esquerda: áreas e páginas;
- centro: prévia em proporção A4;
- direita: propriedades do card selecionado.

5. Geração do PDF

A ação valida se há cards e cria uma nova `ReportVersion` . Hoje é mockado; depois vira chamada real ao backend.

7. Onde entra o Django

A integração futura fica centralizada em `src/services/api.ts` . A ideia é trocar o `mockApi` por um cliente HTTP real sem sair editando componente por componente.

Contrato previsto

- `getAuthenticatedUser` : usuário logado.
- `listWeeklyCycles` : semanas de coleta.
- `listActivityReports` : relatos filtráveis.
- `createActivityReport` : cria relato.
- `uploadPhotos` : envia imagens.
- `getCollectionOverview` : resumo da coleta.

- `generateDraft` : cria relatório editorial.
- `updateReportCard` : edita card editorial.
- `reorderReportCards` : salva ordem manual.
- `removeReportCard` : retira card sem apagar relato.
- `generatePdf` : gera nova versão.
- `getPdfUrl` : retorna URL do PDF.

Variável de ambiente

O arquivo já prevê `NEXT_PUBLIC_API_URL` . Quando a API Django existir, o cliente real deve usar essa URL como base.

8. Permissões

- **Gestor:** cria e consulta relatos; não edita mosaico.
- **Gerente:** seleciona relatos, edita cards e gera PDF.
- **Administrador técnico:** acessa administração de usuários, áreas e prazos.

A proteção visual usa `RoleGuard` . No mundo real, o backend ainda precisa validar tudo. Frontend escondendo botão não é segurança; é só educação visual.

9. Arquivos importantes

- `src/types.ts` : vocabulário do sistema.
- `src/services/api.ts` : interface da API e mock atual.
- `src/services/mock-data.ts` : massa simulada em português.
- `src/components/activity/ActivityForm.tsx` : cadastro de atividade.
- `src/components/activity/activitySchema.ts` : validação Zod.
- `src/app/relatorio-semanal/selecao/page.tsx` : seleção editorial.
- `src/app/relatorio-semanal/[id]/editar/page.tsx` : editor do mosaico.
- `src/components/report/ReportCanvas.tsx` : área A4 com dnd-kit.
- `src/test/domain.test.ts` : testes de regras centrais.

10. Testes existentes

Os testes estão em `src/test/domain.test.ts` . Eles cobrem:

- validação do formulário de atividade;
- bloqueio sem foto principal;

- permissões por perfil;
- seleção de relatos;
- botão de rascunho desabilitado sem seleção;
- reordenação de cards;
- preservação do relato original;
- retirada de card sem excluir relato;
- geração de nova versão;
- fluxo completo de ponta a ponta.

11. Checklist para mexer sem quebrar

- Não coloque `fetch` direto em componente.
- Não edite `ActivityReport` quando a ação for editorial.
- Não crie status de tarefa.
- Não transforme a home em dashboard cheio de gráfico.
- Não invente marca, slogan ou logo.
- Use sempre o nome **synthesis**.
- Antes de entregar, rode `npm run typecheck`, `npm run lint`, `npm test` e `npm run build`.

12. Tradução backendês

<code>ActivityReport</code>	= tabela de entrada, fonte da verdade
<code>ReportCard</code>	= DTO/editorial snapshot para o relatório
<code>WeeklyReport</code>	= aggregate root do relatório semanal
<code>ReportSection</code>	= agrupamento por área
<code>ReportVersion</code>	= artefato gerado, imutável
<code>mockApi</code>	= fake repository/client
<code>TanStack Query</code>	= cache client-side + loading/error states
<code>RoleGuard</code>	= autorização visual, não segurança real

Resumo final: o frontend está montado para trocar mock por Django sem precisar sair catando chamada HTTP perdida pela interface. O cérebro pode descansar por cinco minutos.